

Problem List<>

Submit

880

Premium

DescriptionEditorialSolutionsSubmissions

53. Maximum Subarray

Solved

MediumTopicsCompanies

Given an integer array `nums`, find the `subarray` with the largest sum, and return its sum.

Example 1:
Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:
Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:
Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the `divide and conquer` approach, which is more subtle.

GoDaddyGet email that looks so professional as your business is

Code

JavaAuto

```
1 class Solution {
2     public int maxSubArray(int[] nums) {
3         int currentSum = nums[0];
4         int maxSum = nums[0];
5
6         for (int i = 1; i < nums.length; i++) {
7             currentSum = Math.max(nums[i], currentSum + nums[i]);
8             maxSum = Math.max(maxSum, currentSum);
9         }
10
11         return maxSum;
12     }
13 }
```

Saved

Ln 1, Col 1

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3

Input
nums =
[-2,1,-3,4,-1,2,1,-5,4]

Output
6

Expected
6

Problem

Submissions

Leaderboard

Discussions

Editorial

Two children, Lily and Ron, want to share a chocolate bar. Each of the squares has an integer on it.

Lily decides to share a contiguous segment of the bar selected such that:

- The length of the segment matches Ron's birth month, and,
- The sum of the integers on the squares is equal to his birth day.

Determine how many ways she can divide the chocolate.

Example

$s = [2, 2, 1, 3, 2]$

$d = 4$

$m = 2$

Lily wants to find segments summing to Ron's birth day, $d = 4$ with a length equaling his birth month, $m = 2$. In this case, there are two segments meeting her criteria: $[2, 2]$ and $[1, 3]$.

Function Description

Complete the birthday function in the editor below.

birthday has the following parameter(s):

- int $s[n]$: the numbers on each of the squares of chocolate
- int d : Ron's birth day
- int m : Ron's birth month

Returns

- int: the number of ways the bar can be divided

Input Format

The first line contains an integer n , the number of squares in the chocolate bar.

The second line contains n space-separated integers $s[i]$, the numbers on the chocolate squares where $0 \leq i < n$.

The third line contains two space-separated integers, d and m , Ron's birth day and his birth month.

Constraints

- $1 \leq n \leq 100$
- $1 \leq s[i] \leq 5$, where $(0 \leq i < n)$
- $1 \leq d \leq 31$
- $1 \leq m \leq 12$

Sample Input 0

```
15 * The function accepts following parameters:
16 * 1. INTEGER_ARRAY s
17 * 2. INTEGER d
18 * 3. INTEGER m
19 */
20
21 public static int birthday(List<Integer> s, int d, int m) {
22
23     int count = 0;
24     int sum = 0;
25
26     for (int i = 0; i < m; i++) {
27         sum += s.get(i);
28     }
29
30     if (sum == d) {
31         count++;
32     }
33
34     for (int i = m; i < s.size(); i++) {
35         sum += s.get(i);
36         sum -= s.get(i - m);
37     }
38
39     if (sum == d) {
40         count++;
41     }
42 }
```

Line: 58 Col: 1

☐ Test against custom input

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

	Input (stdin)	
☒ Sample Test case 0	5	Download
☒ Sample Test case 1	1 2 1 3 2	
☒ Sample Test case 2	3 2	

Problem

Submissions

Leaderboard

Discussions

Editorial

We define subsequence as any subset of an array. We define a subarray as a contiguous subsequence in an array.
Given an array, find the maximum possible sum among:

- 1. all nonempty subarrays.
- 2. all nonempty subsequences.

Print the two values as space-separated integers on one line.

Note that empty subarrays/subsequences should not be considered.

Example

`arr = [-1, 2, 3, -4, 5, 10]`

The maximum subarray sum is comprised of elements at indices `[1 - 5]`. Their sum is `2 + 3 + -4 + 5 + 10 = 16`.

The maximum subsequence sum is comprised of elements at indices `[1, 2, 4, 5]` and their sum is

`2 + 3 + 5 + 10 = 20`.

Function Description

Complete the `maxSubarray` function in the editor below.

`maxSubarray` has the following parameter(s):

- `int arr[n]`: an array of integers.

Returns

- `int[2]`: the maximum subarray and subsequence sums.

Input Format

The first line of input contains a single integer `t`, the number of test cases.

The first line of each test case contains a single integer `n`.

The second line contains `n` space-separated integers `arr[i]` where $0 \leq i < n$.

Constraints

- $1 \leq t \leq 10$
- $1 \leq n \leq 10^5$
- $-10^4 \leq arr[i] \leq 10^4$

The subarray and subsequences you consider should have at least one element.

Sample Input 0

2

```
32 if (value > 0) {
33     maxSubsequence += value;
34 }
35 }
36
37 // If all elements are negative, choose the largest element
38 if (maxSubsequence <= 0) {
39     maxSubsequence = Collections.max(arr);
40 }
41
42 List<Integer> result = new ArrayList<>();
43 result.add(maxSubarray);
44 result.add(maxSubsequence);
45
46 return result;
47 }
48
49
50
51
52
53
54
55
56 public class Solution {
57     public static void main(String[] args) throws IOException {
58         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
59         BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
60     }
61 }
```

Line: 54 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

Sample Test case 1

2

DescriptionEditorsSolutionsSubmissions

918. Maximum Sum Circular Subarray

MediumTopicsCompaniesHint

Given a **circular integer array** `nums` of length `n`, return the *maximum possible sum of a non-empty subarray of `nums`*.

A **circular array** means the end of the array connects to the beginning of the array. Formally, the next element of `nums[i]` is `nums[(i + 1) % n]` and the previous element of `nums[i]` is `nums[(i - 1 + n) % n]`.

A **subarray** may only include each element of the fixed buffer `nums` at most once. Formally, for a subarray `nums[i], nums[i + 1], ..., nums[j]`, there does not exist `1 ≤ k1, k2 ≤ j` with `k1 % n == k2 % n`.

Example 1:

Input: `nums = [1,-2,3,-2]`

Output: `3`

Explanation: Subarray `[3]` has maximum sum `3`.

Example 2:

Input: `nums = [5,-3,5]`

Output: `10`

Explanation: Subarray `[5,5]` has maximum sum `5 + 5 = 10`.

Example 3:

Input: `nums = [-3,-2,-3]`

Output: `-2`

Explanation: Subarray `[-2]` has maximum sum `-2`.

Constraints:

- `n == nums.length`
- `1 ≤ n ≤ 3 * 104`
- `-2 * 104 ≤ nums[i] ≤ 2 * 104`

Code

JavaAuto

```
12
13
14 // Minimum subarray sum
15 currentMin = Math.min(nums, currentMin + num);
16 minSum = Math.min(minSum, currentMin);
17
18 totalSum += num;
19 }
20
21 // If all elements are negative
22 if (maxSum < 0) {
23     return maxSum;
24 }
25
26 // Maximum of normal subarray and circular subarray
27 return Math.max(maxSum, totalSum - minSum);
28 }
```

SavedLn 28, Col 2

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3

Input

nums =

[1,-2,3,-2]

Output

3

Expected

3

Contribute a Testcase

Problem List

8. String to Integer (atoi)

Medium

Topics

Companies

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace:** Ignore any leading whitespace (`" "`).

2. **Signedness:** Determine the sign by checking if the next character is `['+', '-']` or `['+', '-']`, assuming positivity if neither present.

3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.

4. **Rounding:** If the integer is out of the 32-bit signed integer range `[-231, 231 - 1]`, then round the integer to remain in the range. Specifically, integers less than `-231` should be rounded to `-231`, and integers greater than `231 - 1` should be rounded to `231 - 1`.

Return the integer as the final result.

Example 1:

Input: `s = "42"`

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader position.

Step 1: `"42"` (no characters read because there is no leading whitespace)

Step 2: `"42"` (no characters read because there is neither a `'+'` nor `'-'`)

Step 3: `"42"` (`"42"` is read in)

Example 2:

Input: `s = " -042"`

Output: -42

6.4K

992

114 Online

Code

Java

Auto

```
23 result = result * 10 + (s.charAt(i) - '0');
24
25 // Step 4: Check overflow
26 if (sign * result > Integer.MAX_VALUE) {
27     return Integer.MAX_VALUE;
28 }
29
30 if (sign * result < Integer.MIN_VALUE) {
31     return Integer.MIN_VALUE;
32 }
33
34 i++;
35 }
36
37 return (int) (sign * result);
38
39 }
```

Saved

Ln 38, Col 2

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Case 4

Case 5

Input

`s =`

`"42"`

Output

42

Expected

42

Contribute a testcase

Problem

Submissions

Leaderboard

Discussions

Editorial

You are given a string containing characters *A* and *B* only. Your task is to change it into a string such that there are no matching adjacent characters. To do this, you are allowed to delete zero or more characters in the string.

Your task is to find the minimum number of required deletions.

Example

s = *AABAAB*

Remove an *A* at positions 0 and 3 to make *s* = *ABAB* in 2 deletions.

Function Description

Complete the `alternatingCharacters` function in the editor below.

`alternatingCharacters` has the following parameter(s):

- `string s`: a string

Returns

- `int`: the minimum number of deletions required

Input Format

The first line contains an integer *q*, the number of queries.

The next *q* lines each contain a string *s* to analyze.

Constraints

- $1 \leq q \leq 10$
- $1 \leq \text{length of } s \leq 10^5$
- Each string *s* will consist only of characters *A* and *B*.

Sample Input

```
5
AAAA
BBBBB
ABABABAB
BABABA
AAABBB
```

Sample Output

```
3
4
```

```
27
18 public static int alternatingCharacters(String s) {
19     int deletions = 0;
20
21     for (int i = 1; i < s.length(); i++) {
22         if (s.charAt(i) == s.charAt(i - 1)) {
23             deletions++;
24         }
25     }
26
27     return deletions;
28
29 }
30
31
32 }
33
34 public class Solution {
35     public static void main(String[] args) throws IOException {
36         BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
37         BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));
38
39         int q = Integer.parseInt(bufferedReader.readLine().trim());
40
41         for (int qItr = 0; qItr < q; qItr++) {
42             String s = bufferedReader.readLine();
43
44             int result = Result.alternatingCharacters(s);
45
46             bufferedWriter.write(result + "\n");
47         }
48         bufferedWriter.close();
49     }
50 }
```

Line: 27 Col: 18

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

Download

Sample Test case 1

1 5

Problem List

Submit

Premium

DescriptionEditorialSolutionsSubmissions

Explanation: The answer is "b", with the length of 1.

Example 3:
Input: s = "pwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:
• 0 <= s.length <= 10⁵
• s consists of English letters, digits, symbols and spaces.

Seen this question in a real interview before? 1/5

Accepted 10,145,860 / 25.3M Acceptance Rate 39.8%

TopicsCompaniesHint 1Similar QuestionsDiscussion (932)

Copyright © 2025 LeetCode. All rights reserved.

Code

```
1  int index[] = new int[26];
2  int max = 0;
3  for (int right = 0; right < s.length(); right++) {
4      char ch = s.charAt(right);
5      if (index[ch] >= left) {
6          left = index[ch] + 1;
7      }
8      index[ch] = right;
9      max = Math.max(max, right - left + 1);
10 }
11 return max;
```

SavedLn 23, Col 2

TestcaseTest Result

AcceptedRuntimes: 0 ms

Case 1Case 2Case 3

Input
s = "abcabcbb"

Output
3

Expected
3

Contribute a testcase

Problem List

890. Find and Replace Pattern

Description

Editorial

Solutions

Submissions

890. Find and Replace Pattern

Medium

Topics

Companies

Given a list of strings `words` and a string `pattern`, return a list of `words[i]` that match `pattern`. You may return the answer in **any order**.

A word matches the pattern if there exists a permutation of letters `p` so that after replacing every letter `x` in the pattern with `p(x)`, we get the desired word.

Recall that a permutation of letters is a bijection from letters to letters: every letter maps to another letter, and no two letters map to the same letter.

Example 1:

Input: `words = ["abc","deq","mee","aqq","dkd","ccc"], pattern = "abb"`
Output: `["mee","aqq"]`
Explanation: "mee" matches the pattern because there is a permutation (a -> m, b -> e, ...) . "ccc" does not match the pattern because (a -> c, b -> c, ...) is not a permutation, since a and b map to the same letter.

Example 2:

Input: `words = ["a","b","c"], pattern = "a"`
Output: `["a","b","c"]`

Constraints:

- `1 <= pattern.length <= 20`
- `1 <= words.length <= 50`
- `words[i].length == pattern.length`
- `pattern` and `words[i]` are lowercase English letters.

Code

Java

Auto

```
20 for (int i = 0; i < word.length(); i++) {
21     char p = pattern.charAt(i);
22     char w = word.charAt(i);
23
24     if (!map1.containsKey(p) || map1.get(p) != w)
25         return false;
26
27     if (!map2.containsKey(w) || map2.get(w) != p)
28         return false;
29
30     map1.put(p, w);
31     map2.put(w, p);
32 }
33
34 return true;
35 }
36 }
```

Saved

Ln 36, Col 2

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

words =

["abc","deq","mee","aqq","dkd","ccc"]

pattern =

"abb"

Output

["mee","aqq"]

Expected

["mee","aqq"]

4K

28

8 Online

Google Gemini

Gemini for 1 year at no cost

Sign Up

Problem List

1408. String Matching in an Array

EasyTopicsCompaniesHint

Given an array of string words, return all strings in words that are a **substring** of another word. You can return the answer in **any order**.

Example 1:
Input: words = ["mass","as","hero","superhero"]
Output: ["as","hero"]
Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".
"hero","as" is also a valid answer.

Example 2:
Input: words = ["leetcode","et","code"]
Output: ["et","code"]
Explanation: "et", "code" are substring of "leetcode".

Example 3:
Input: words = ["blue","green","bu"]
Output: []
Explanation: No string of words is substring of another string.

Constraints:

- 1 <= words.length <= 100
- 1 <= words[i].length <= 30
- words[i] contains only lowercase English letters.
- All the strings of words are **unique**.

Get email that looks as professional as your business n

15K17310 Online

Code

JavaAuto

```
2
3 class Solution {
4     public List<String> stringMatching(String[] words) {
5         List<String> result = new ArrayList<>();
6
7         for (int i = 0; i < words.length; i++) {
8             for (int j = 0; j < words.length; j++) {
9                 if (i != j && words[j].contains(words[i])) {
10                     result.add(words[i]);
11                     break;
12                 }
13             }
14         }
15
16         return result;
17     }
18 }
```

SavedLn 18, Col 2

TestcaseTest Result

AcceptedRuntime: 0 ms

Case 1Case 2Case 3

Input

words =
["mass","as","hero","superhero"]

Output

["as","hero"]

Expected

["as","hero"]

Contribute a testcase

eg

Search...

Courses

Tutorials

Practice

Jobs

Problem

Editorial

Submissions

Comments

Output Window

Compilation Results

Custom Input

Y.O.G.I. (AI Bot)

Problem Solved Successfully

Test Cases Passed

1111 / 1111

Attempts : Correct / Total

1 / 2

Accuracy : 50%

Points Scored

8 / 8

Your Total Score: 8

Time Taken

0.03

Submit Feedback

C++ (17)

Start Timer

```
1 class Solution {
2 public:
3     vector<int> search(string &pat, string &txt) {
4         vector<int> result;
5         int m = pat.size();
6         int n = txt.size();
7
8         // Create LPS array
9         vector<int> lps(m, 0);
10
11         int len = 0;
12         int i = 1;
13
14         while (i < m) {
15             if (pat[i] == pat[len]) {
16                 len++;
17                 lps[i] = len;
18                 i++;
19             } else {
20                 if (len != 0) {
21                     len = lps[len - 1];
22                 } else {
23                     lps[i] = 0;
24                     i++;
25                 }
26             }
27         }
28
29         // KMP Search
30         i = 0;
31         int j = 0;
32
33         while (i < n) {
34             if (txt[i] == pat[j]) {
35                 i++;
36                 j++;
37             }
38
39             if (j == m) {
40                 result.push_back(i - j);
41                 j = lps[j - 1];
42             } else if (i < n && txt[i] != pat[j]) {
43                 if (j != 0) {
44                     j = lps[j - 1];
45                 } else {
46                     i++;
47                 }
48             }
49         }
50
51         return result;
52     }
```

Custom Input

Compile & Run

Submit